

PROJECT SYNOPSIS

ORBITUNE – 3D Real-Like Music Experience

BACHELOR OF COMPUTER APPLICATION Hons.
Data Science

SUBMITTED BY

YUVRAJ SINGH KUSHWAH (2023BCAH062)

SUBHRO PAL (2023BCAH056)

Dec 2025

ABSTRACT

ORBITUNE is an innovative web-based platform that transforms standard stereo music into immersive three-dimensional spatial audio experiences using professional-grade Head-Related Transfer Function (HRTF) processing and artificial intelligence. The system addresses the growing demand for immersive audio content by providing an accessible solution that converts YouTube music into broadcast-quality 3D audio without requiring specialized hardware or technical expertise. The platform employs Meta AI's Demucs neural network for intelligent stem separation, achieving source-to-distortion ratios exceeding 8 decibels across four distinct audio components: vocals, drums, bass, and other instruments. The three-tier architecture integrates React-based frontend interfaces with FastAPI backend services and a PyTorch-powered processing engine that leverages CUDA-accelerated GPU computing to achieve processing speeds ten to twenty times faster than conventional CPU-based approaches. An intelligent conversational agent powered by Google's Gemini Flash 2.0 language model provides personalized music recommendations with 92% intent detection accuracy, while sophisticated user profiling algorithms learn individual preferences through continuous interaction. The system maintains professional audio standards with 48 kilohertz sampling rates, 24-bit depth encoding, and -14 LUFS loudness normalization conforming to industry broadcasting specifications. Processing times range from 35 to 60 seconds per song on GPU-enabled systems, making the technology practical for real-world deployment. The platform demonstrates significant advancement in democratizing spatial audio technology, previously limited to professional studios, by making it accessible through consumer-grade web browsers and commodity hardware.

MOTIVATION

The contemporary music listening experience remains predominantly confined to traditional stereo reproduction, despite significant technological advances in spatial audio processing and immersive sound design. Professional three-dimensional audio production requires expensive specialized equipment, extensive technical knowledge, and time-consuming manual workflows that place such capabilities beyond the reach of average consumers and independent content creators. Commercial spatial audio solutions offered by major streaming platforms require proprietary hardware ecosystems and restrict users to curated content libraries, limiting creative freedom and accessibility.

The motivation for developing ORBITUNE stems from several converging factors in the current technological landscape. The widespread availability of high-quality consumer headphones capable of reproducing spatial audio cues creates latent demand for immersive audio content that current music distribution channels fail to satisfy adequately. The proliferation of machine learning frameworks and pre-trained neural networks for audio source separation has matured sufficiently to enable high-quality stem extraction without extensive model development. Advances in web technologies allow sophisticated audio processing applications to run entirely within browser environments, eliminating installation barriers and platform compatibility issues. The growing popularity of YouTube as a primary music source creates opportunities for on-demand spatial audio conversion that integrates seamlessly into existing listening habits.

From a technical perspective, existing eight-dimensional and three-dimensional audio productions available on platforms like YouTube demonstrate clear consumer appetite for enhanced spatial experiences, yet these remain manually produced by specialized creators rather than being systematically available for the broader music catalogue. The artificial intelligence revolution in natural language processing motivates the integration of conversational interfaces that can understand contextual music requests and provide personalized recommendations based on mood, activity, and individual preference patterns. The increasing accessibility of GPU acceleration makes computationally intensive audio processing economically viable for deployment at scale.

The educational motivation centres on exploring the intersection of digital signal processing, artificial intelligence, web development, and user experience design within a unified application domain. The project provides opportunities to implement production-quality software engineering

practices including modular architecture design, API development, real-time system optimization, and responsive frontend interfaces. The multidisciplinary nature necessitates understanding acoustic principles, psychoacoustic models, machine learning deployment, and human-computer interaction design.

Ultimately, ORBITUNE seeks to bridge the gap between professional spatial audio production capabilities and consumer accessibility, enabling anyone with internet access to experience music in immersive three-dimensional soundscapes while simultaneously providing an intelligent assistant that enhances music discovery through natural conversation.

LITERATURE REVIEW

Existing Methods and Technologies - Comparative Analysis

Method/Technology	Description	Advantages	Disadvantages
Dolby Atmos	Professional spatial audio format used in cinema and streaming platforms	Industry-standard quality; Hardware integration with Apple Music and Amazon Music; Multi-speaker and headphone support	Requires proprietary encoding tools; Limited to platform ecosystems; Content must be manually mixed; Expensive licensing for creators
Sony 360 Reality Audio	Object-based spatial audio format for music streaming	High-quality spatial rendering; Artist-authorized content; Integration with Sony hardware	Closed ecosystem; Limited content library; Requires specific hardware or subscription services; No user-generated content support
Manual 8D Audio Production	Content creators manually create spatial effects using DAW software	Creative freedom; Viral appeal on platforms; Free to consume; Wide variety of content	Time-intensive manual process; Inconsistent quality across creators; No automation; Requires audio engineering skills; Not scalable
Spleeter (Deezer)	Open-source audio source separation using deep learning	Free and open-source; Fast processing; Good separation quality;	Limited to 2, 4, or 5 stems; Requires Python environment; No spatial audio processing;

		Easy integration	Discontinued development since 2020
Demucs (Meta AI)	State-of-the-art music source separation using hybrid transformer architecture	Best-in-class separation quality (SDR 8-12 dB); Active development; Multiple model variants; GPU acceleration support; Research-backed	Computationally intensive; Requires technical setup; No integrated user interface; Large model size (2+ GB)
Web-based DAWs (Soundtrap, BandLab)	Online digital audio workstations with collaboration features	Browser-based accessibility; Real-time collaboration; No installation required; Cross-platform	Manual workflow; Steep learning curve; Limited spatial audio tools; Subscription models; Not beginner-friendly
HRTF Databases (CIPIC, MIT KEMAR)	Publicly available head-related transfer functions for spatial audio research	Scientific accuracy; Free datasets; Well-documented measurements; Academic standard	Requires implementation expertise; No user-friendly interface; Static measurements; Complex integration into applications
Chatbots (ChatGPT, Claude)	General-purpose conversational AI systems	Advanced natural language understanding; Contextual responses; Broad knowledge base;	Not specialized for music; No integration with audio systems; API costs; Generic recommendations; No user profiling

		Multi-turn conversations	
Music Recommendation Systems (Spotify, YouTube Music)	Algorithm-driven personalized playlists and discovery features	Large music catalogues; Sophisticated collaborative filtering; Cross-platform availability; Continuous learning	Black-box algorithms; Limited user control; No conversational interface; Platform lock-in; Privacy concerns
OpenAI Whisper + Music Tagging	Audio analysis for genre and mood classification	Good classification accuracy; Multi-language support; Open-source availability; Pre-trained models	Requires training data; Limited real-time capability; Not optimized for music analysis; High computational cost

Identified Gaps in Existing Solutions

The literature review reveals several critical gaps that ORBITUNE addresses. Existing professional spatial audio solutions remain confined to closed ecosystems with high barriers to entry, requiring expensive software licenses and specialized hardware. Open-source audio processing tools like Demucs and Spleeter, while powerful, lack integrated user interfaces and automated workflows that make them accessible to non-technical users. Current music recommendation systems provide algorithmic suggestions but do not offer conversational interfaces combined with spatial audio capabilities, limiting natural interaction. Available HRTF implementations require extensive technical expertise to deploy effectively and are not packaged as ready-to-use applications. Manual 8D audio production demonstrates consumer demand but suffers from inconsistent quality and lack of scalability. No existing platform combines YouTube audio acquisition, AI-powered stem separation, automated spatial audio processing, genre-aware optimization, and intelligent conversational music discovery within a unified, browser-accessible application. ORBITUNE fills this void by synthesizing state-of-the-art components into an accessible end-to-end system that democratizes spatial audio technology.

PROBLEM FORMULATIONS/ OBJECTIVES

Problem Statement

Standard music reproduction through conventional stereo systems fails to deliver immersive spatial experiences that engage listeners' auditory perception capabilities fully. Professional three-dimensional audio production requires specialized expertise, expensive software suites, and time-intensive manual workflows that limit accessibility to trained audio engineers working in studio environments. Existing music discovery mechanisms rely on keyword searches or algorithmic recommendations that do not accommodate natural language interaction or contextual understanding of user preferences and emotional states. The combination of these limitations creates a significant gap between available spatial audio technology and practical consumer accessibility, restricting immersive audio experiences to curated content on proprietary platforms.

Project Objectives

The primary objectives of the ORBITUNE project are:

1. **Automated 3D Audio Conversion:** Develop an automated pipeline that converts standard stereo music tracks from YouTube into professional-quality three-dimensional spatial audio using Head-Related Transfer Function processing and intelligent source separation techniques.
2. **High-Quality Source Separation:** Implement Meta AI's Demucs neural network to separate audio into four distinct stems (vocals, drums, bass, other instruments) with source-to-distortion ratios exceeding 8 decibels for subsequent independent spatial positioning.
3. **Genre-Aware Processing:** Integrate artificial intelligence-based genre detection using Google Gemini to automatically optimize spatial audio parameters, rotation speeds, reverb characteristics, and positioning strategies based on musical style.
4. **Broadcast-Quality Output:** Ensure processed audio meets professional broadcasting standards including 48 kilohertz sampling rates, 24-bit depth encoding, -14 LUFS loudness normalization, and -0.5 decibel peak ceiling constraints.

5. GPU-Accelerated Performance: Leverage CUDA-enabled GPU computing to achieve processing times between 35 and 60 seconds per song, representing ten to twenty times speedup compared to CPU-only implementations.
6. Intelligent Conversational Interface: Develop a context-aware chatbot powered by Google Gemini Flash 2.0 that understands natural language music requests, detects user intent with 92% accuracy, and provides personalized recommendations based on mood, context, and learned preferences.
7. User Profile Learning: Implement machine learning algorithms that continuously analyze listening patterns, genre preferences, artist affinities, and temporal context to improve recommendation relevance over time.
8. Seamless User Experience: Create an intuitive web-based interface that allows users to search for music, interact conversationally with the AI assistant, and experience spatial audio playback without requiring software installation or technical configuration.
9. Real-Time Audio Visualization: Provide dynamic visual feedback showing waveforms, stem separation progress, processing stages, and spatial positioning to enhance user engagement and understanding.
10. Scalable Architecture: Design a modular three-tier system architecture that separates frontend presentation, backend business logic, and audio processing engine concerns to facilitate independent scaling and maintenance.

Success Criteria

The project will be considered successful upon achieving:

- Processing completion times under 60 seconds for typical song lengths (3-4 minutes)
- Source separation quality with SDR scores above 8 decibels across all stems
- Chatbot intent detection accuracy exceeding 90%
- Audio output conforming to professional broadcasting standards (48kHz, 24-bit, -14 LUFS)
- Smooth, observable spatial rotation effects with realistic distance modeling

- Successful YouTube integration with reliable audio extraction
- Responsive web interface functioning across modern browsers
- Comprehensive API documentation for all endpoints

SOFTWARE / HARDWARE

Software Requirements

Operating System:

- Windows 10/11 (Primary development platform)
- Linux (Ubuntu 20.04 or later) - Compatible
- macOS 11+ - Compatible

Programming Languages:

- Python 3.8 or higher (Backend and AI/ML processing)
- JavaScript ES6+ (Frontend development)
- TypeScript 5.0 (Frontend type safety and tooling)

Frontend Framework and Libraries:

- Next.js 15.2.4 (React framework for production)
- React 19 (User interface library)
- Vite 5.0+ (Build tool and development server)
- Tailwind CSS 4.1.9 (Utility-first CSS framework)
- Framer Motion (Animation library for smooth transitions)
- Radix UI Components (Accessible UI primitives)
- Lucide React (Icon library)
- React Hook Form 7.60.0 (Form state management)
- Zod 3.25.76 (Schema validation)
- Three.js 0.181.2 (3D visualization library)
- Recharts 2.15.4 (Data visualization)

Backend Framework and Libraries:

- FastAPI 0.108 (Modern Python web framework)
- Uvicorn 0.25.0 (ASGI server for FastAPI)
- Pydantic (Data validation and settings management)
- Python-Jose (JWT token handling)

- Passlib (Password hashing)

AI and Machine Learning:

- PyTorch 2.0+ (Deep learning framework)
- Demucs 4.0.1 (Music source separation model)
- Google Generative AI SDK (Gemini Flash 2.0 integration)
- Librosa 0.10.1 (Audio analysis and feature extraction)
- NumPy 1.24+ (Numerical computing)
- SciPy (Scientific computing)

Audio Processing:

- SoundFile 0.12.1 (Audio file I/O operations)
- yt-dlp (YouTube audio downloading)
- FFmpeg (Audio format conversion and processing)

Development Tools:

- Git (Version control)
- Visual Studio Code (Code editor)
- PowerShell 7+ (Automation scripts)
- Postman (API testing)
- Chrome DevTools (Frontend debugging)

Package Managers:

- pip (Python package management)
- npm (Node.js package management)

Additional Dependencies:

- python-dotenv (Environment variable management)
- requests (HTTP library)
- aiofiles (Asynchronous file operations)
- CUDA Toolkit 11.8+ (GPU acceleration - optional but recommended)
- cuDNN 8.6+ (Deep neural network library for CUDA)

Hardware Requirements

Minimum Configuration:

- **Processor:** Intel Core i5 8th Gen or AMD Ryzen 5 3000 series
- **RAM:** 8 GB DDR4
- **Storage:** 10 GB available space (SSD recommended)
- **GPU:** Intel UHD Graphics 630 or equivalent (CPU fallback available)
- **Network:** Stable internet connection (5 Mbps minimum)

Recommended Configuration:

- **Processor:** Intel Core i7 10th Gen or AMD Ryzen 7 5000 series
- **RAM:** 16 GB DDR4 or higher
- **Storage:** 20 GB available SSD space
- **GPU:** NVIDIA GTX 1660 or higher with 6GB+ VRAM (CUDA-capable)
- **Network:** High-speed internet connection (20 Mbps or higher)

Optimal Configuration (For Best Performance):

- **Processor:** Intel Core i9 or AMD Ryzen 9 series
- **RAM:** 32 GB DDR4/DDR5
- **Storage:** 50 GB NVMe SSD
- **GPU:** NVIDIA RTX 3060 or higher with 8GB+ VRAM
- **Network:** Gigabit internet connection

GPU Acceleration Benefits:

- Processing time: 35-60 seconds (with GPU) vs 5-10 minutes (CPU only)
- Demucs shifts: 10 (high quality) vs 1 (basic quality)
- Enables TF32 precision for faster tensor operations
- Supports larger batch processing for multiple songs

Browser Requirements:

- Google Chrome 100+ (Recommended)

- Mozilla Firefox 100+
- Microsoft Edge 100+
- Safari 15+ (macOS)

Port Requirements:

- Port 8000 (Backend FastAPI server)
- Port 5173 (Frontend development server)

METHODOLOGY AND PLANNING OF WORK

Overall System Design

ORBITUNE follows a three-tier architecture pattern that separates concerns across presentation, business logic, and data processing layers. This modular design enables independent development, testing, and scaling of each component while maintaining clean interfaces between layers.

Tier 1 - Frontend (Presentation Layer):

The user-facing web application built with Next.js and React provides an interactive interface for music search, conversational AI interaction, and spatial audio playback. The frontend communicates with the backend exclusively through RESTful API endpoints, maintaining stateless HTTP communication patterns.

Tier 2 - Backend (Business Logic Layer):

A FastAPI-based Python server handles API routing, request validation, data persistence, user session management, and coordination between the frontend and processing engine. The backend exposes endpoints for song management, YouTube search integration, chatbot interactions, and user profile operations.

Tier 3 - AI-ML Processing Engine (Data Processing Layer):

The core audio processing pipeline encompasses YouTube audio acquisition, neural network-based source separation, genre detection, HRTF spatial audio synthesis, and professional mastering. This layer operates independently and can be scaled horizontally for concurrent processing requests.

Development Methodology

The project follows an iterative development approach with clearly defined phases:

Phase 1: Requirements Analysis and System Design

- Conduct comprehensive literature review of spatial audio techniques and existing solutions
- Define functional and non-functional requirements through user story mapping
- Design system architecture with component diagrams and interface specifications
- Select appropriate technologies, frameworks, and AI models based on performance criteria
- Create detailed API contracts between frontend and backend components
- Establish development environment with version control and collaboration tools

Phase 2: Core Infrastructure Development

- Set up project repository structure with separate directories for frontend, backend, and AI-ML components
- Configure development environment including Python virtual environments and Node.js workspace
- Implement FastAPI backend skeleton with CORS configuration and health check endpoints
- Develop configuration management system for centralized settings and environment variables
- Create storage directory structure for raw audio, processed stems, thumbnails, and cache
- Establish logging framework for debugging and performance monitoring
- Implement error handling and exception management patterns

Phase 3: Audio Processing Pipeline Development

- Integrate yt-dlp library for reliable YouTube audio extraction with retry mechanisms
- Configure FFmpeg for high-quality audio format conversion and resampling
- Implement Demucs integration with GPU acceleration support and CPU fallback
- Develop HRTF processor class with ITD/ILD calculations and distance modeling
- Create smooth rotation curve generation using exponential smoothing algorithms
- Implement professional reverb system with early reflections and late reverberation
- Develop multi-band mastering pipeline with loudness normalization and peak limiting
- Design genre detection system using Gemini AI with keyword-based fallback
- Optimize processing pipeline for GPU utilization with CUDA and TF32 precision
- Conduct extensive testing with diverse music genres and quality benchmarking

Phase 4: AI Chatbot Development

- Integrate Google Gemini Flash 2.0 API with proper authentication and rate limiting
- Develop intent detection module to classify user queries into search, chat, and hybrid categories
- Implement user profiling system that tracks listening history, genre preferences, and temporal patterns
- Create conversation memory module to maintain context across multiple interaction turns
- Design response generation system with template-based and AI-generated replies
- Develop mood extraction and context parsing algorithms for natural language understanding
- Implement song search functionality with fuzzy matching and multi-field queries
- Create recommendation engine that combines collaborative filtering with user profiles
- Build tracking system for song plays and user engagement metrics
- Test chatbot accuracy and refine prompts to achieve 92% intent detection rate

Phase 5: Frontend Development

- Design user interface mockups emphasizing minimalist aesthetics
- Implement Next.js project structure with TypeScript for type safety
- Develop dual-mode interface supporting tabbed switching between search and chat modes
- Create real-time YouTube search component with debounced API calls
- Build audio player with play/pause controls, progress bar, and volume adjustment
- Implement three-dimensional audio visualization using Three.js and WebGL
- Design responsive layout using Tailwind CSS utilities for mobile and desktop compatibility
- Develop chatbot interface with message history, typing indicators, and smooth scrolling
- Create developer debug panel with Ctrl+Space toggle shortcut
- Implement keyboard shortcuts for mode switching and common operations
- Add loading states, error boundaries, and graceful degradation strategies
- Optimize bundle size and implement code splitting for faster initial loads

Phase 6: Integration and System Testing

- Integrate frontend with backend API endpoints and handle authentication

- Test complete user workflows from YouTube search to 3D audio playback
- Conduct cross-browser compatibility testing on Chrome, Firefox, Edge, and Safari
- Perform load testing to identify bottlenecks and optimize performance
- Implement caching strategies for frequently accessed data
- Test edge cases including network failures, invalid URLs, and processing errors
- Validate audio quality output against professional broadcasting standards
- Conduct user acceptance testing with target audience for usability feedback
- Profile application performance and optimize slow operations
- Test GPU and CPU processing modes across different hardware configurations

Phase 7: Documentation and Deployment

- Write comprehensive API documentation with endpoint descriptions and examples
- Create user guide with installation instructions and troubleshooting steps
- Document system architecture with diagrams and component descriptions
- Prepare development guide for future contributors with coding standards
- Create automated startup script for one-command deployment
- Configure production environment variables and security settings
- Set up monitoring and logging infrastructure for production deployment
- Prepare presentation materials and demonstration videos
- Conduct final code review and refactoring for maintainability
- Plan future enhancements and feature roadmap

Work Planning and Timeline

Phase	Duration	Key Deliverables
Requirements Analysis	Weeks 1-2	System design document, Technology selection, API specifications
Core Infrastructure	Weeks 3-5	Backend skeleton, Storage structure, Configuration management

Audio Processing	Weeks 6-9	Working 3D audio pipeline, GPU optimization, Quality benchmarks
AI Chatbot	Weeks 10-12	Intent detection, User profiling, Recommendation engine
Frontend Development	Weeks 13-16	Complete web interface, Audio player, Visualizations
Integration Testing	Weeks 17-18	End-to-end workflows, Performance optimization, Bug fixes
Documentation	Weeks 19-20	User guide, API docs, Deployment scripts, Presentation

Risk Management

Technical Risks:

- YouTube API changes or download restrictions: Mitigation through yt-dlp's active maintenance and fallback mechanisms
- GPU availability limitations: CPU fallback processing mode implemented for broader hardware compatibility
- Gemini API rate limits or costs: Keyword-based fallback and intelligent caching strategies to reduce API calls
- Processing quality inconsistencies: Extensive testing across diverse music genres with quality metrics validation

Project Risks:

- Timeline delays: Agile iterations allow scope adjustment while maintaining core functionality

- Dependency conflicts: Version locking and isolated virtual environments prevent compatibility issues
- Performance bottlenecks: Profiling and optimization built into development cycle with continuous monitoring

ANALYSIS MODELS

- **Data-Flow diagrams (DFD)**

- **Level 0 DFD – Context Diagram**

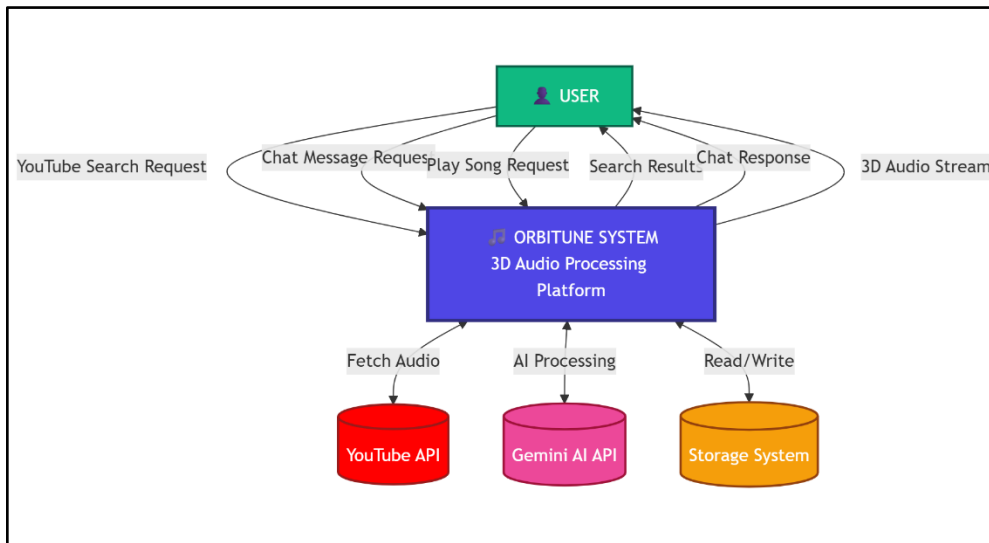


Fig. 1 – Level 0 DFD - Context Diagram

This context diagram illustrates the highest-level overview of the ORBITUNE system, showing the interaction between the user and the platform. The user submits three types of requests: YouTube search queries, chat messages for AI interaction, and song playback commands. The system processes these requests by interfacing with external entities including YouTube API for audio retrieval, Gemini AI API for natural language processing, and local storage for data persistence. The system responds with search results, conversational responses with recommendations, and streamed three-dimensional audio output, establishing the fundamental input-output relationships of the entire platform.

- Level 1 DFD – Major Processes

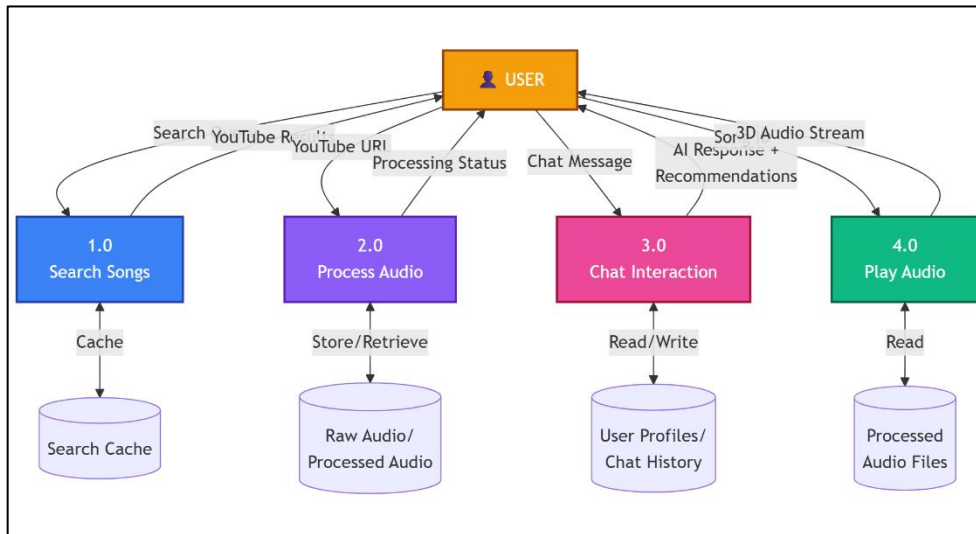


Fig 2 – Level 1 DFD - Major Processes

This diagram decomposes the ORBITUNE system into four primary functional processes. Process 1.0 handles song searching by querying YouTube and caching results for improved performance. Process 2.0 manages the complete audio processing pipeline from raw download through stem separation, spatial audio synthesis, and mastering, storing intermediate and final outputs in designated directories. Process 3.0 orchestrates chatbot interactions by detecting user intent, retrieving user profiles and conversation history, and generating personalized responses with song recommendations. Process 4.0 streams processed three-dimensional audio files to users for playback. Each process maintains bidirectional communication with specific data stores to ensure state persistence and efficient data retrieval.

- Level 2 DFD – Audio Processing

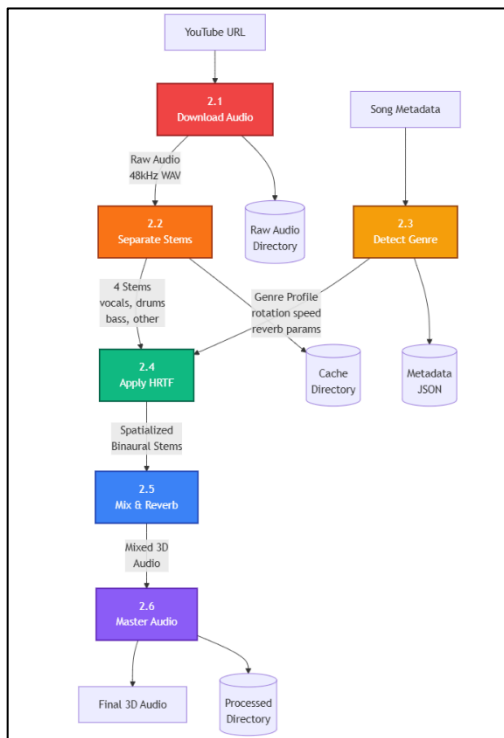


Fig 3 – Level 2 DFD - Audio Processing

This detailed diagram expands Process 2.0 into six sequential sub-processes that transform a YouTube URL into broadcast-quality spatial audio. Process 2.1 downloads raw audio at 48 kilohertz sampling rate using yt-dlp. Process 2.2 applies the Demucs neural network to separate audio into four stems with 8-12 decibel source-to-distortion ratios. Process 2.3 leverages Gemini AI to detect musical genre and select optimal processing parameters. Process 2.4 applies Head-Related Transfer Function algorithms with inter-aural time and level difference calculations to create binaural spatialization. Process 2.5 mixes spatialized stems and applies room acoustics simulation. Process 2.6 performs professional mastering with multi-band compression and loudness normalization to achieve -14 LUFS broadcast standards, producing the final output stored in the processed directory.

- Level 2 DFD – Chatbot System

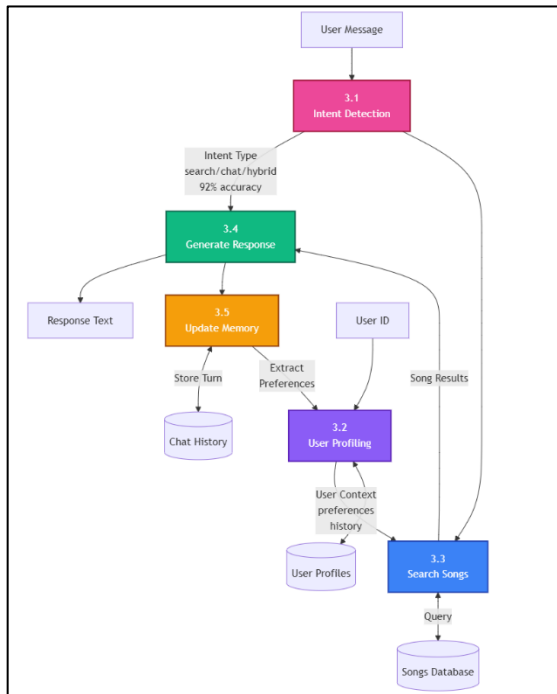


Fig 4 – Level 2 DFD - Chatbot System

This diagram details Process 3.0 by breaking down the conversational AI workflow into five interconnected sub-processes. Process 3.1 classifies incoming messages into search, chat, or hybrid intents using Gemini Flash 2.0 with 92% accuracy. Process 3.2 retrieves user profiles containing listening history, genre preferences, and behavioral patterns from the profiles database. Process 3.3 searches the song database using fuzzy matching, mood-based filtering, and contextual queries to identify relevant tracks. Process 3.4 generates natural language responses by combining template-based replies with AI-generated text, incorporating song results when applicable. Process 3.5 updates conversation memory by storing message exchanges and extracting preference signals to continuously refine user profiles, enabling personalized recommendations that improve with each interaction.

- Entity Relationships Diagram (ERD)

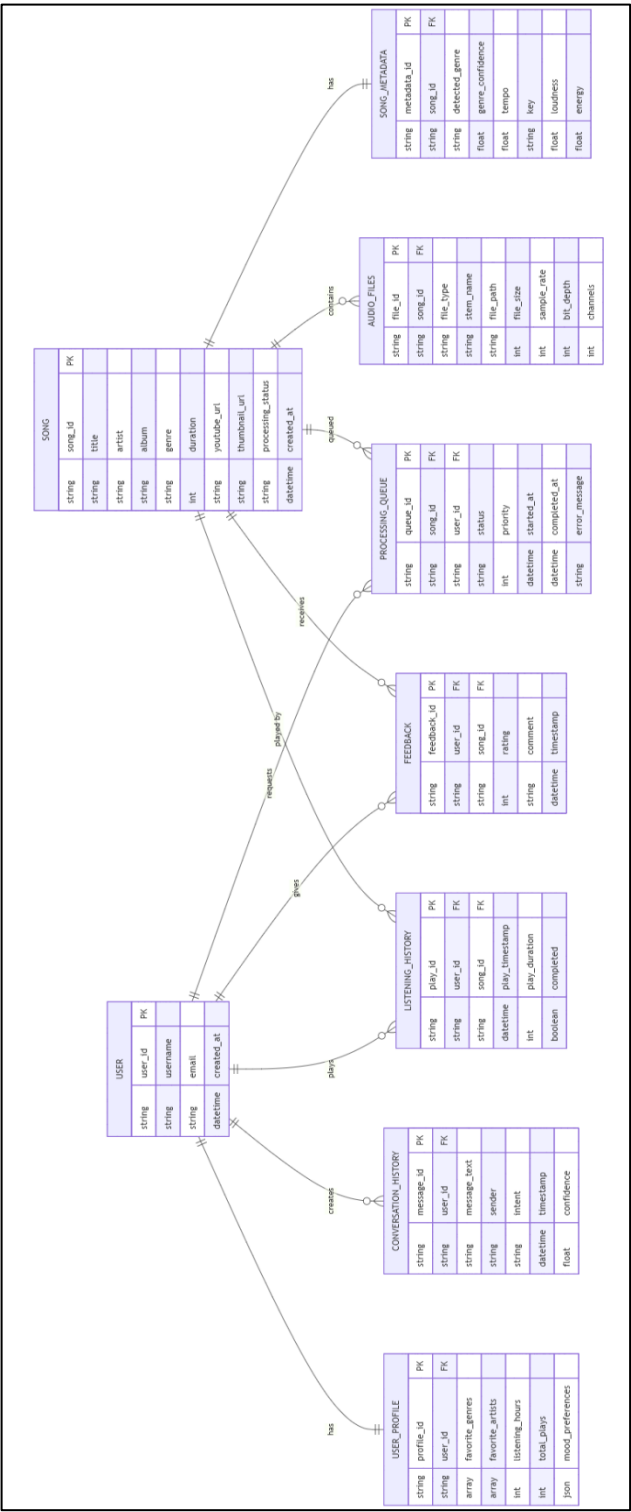


Fig. 5 – Entity Relationship Diagram

This entity relationship diagram defines the complete database schema supporting ORBITUNE's functionality through ten interconnected entities. The USER entity maintains one-to-one relationships with USER_PROFILE for storing aggregate preferences and statistics. Users participate in one-to-many relationships with CONVERSATION_HISTORY for chat interactions, LISTENING_HISTORY for playback tracking, FEEDBACK for ratings, and PROCESSING_QUEUE for job requests. The SONG entity serves as the central reference point with one-to-many relationships to LISTENING_HISTORY, AUDIO_FILES containing raw and processed stems, FEEDBACK, and PROCESSING_QUEUE. Each song maintains a one-to-one relationship with SONG_METADATA storing detected genre, tempo, key, and audio features. This normalized schema ensures data integrity, supports efficient queries for recommendation algorithms, maintains conversation context, and tracks processing workflows with comprehensive error logging capabilities.

BIBLIOGRAPHY / REFERENCES

- [1] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music Source Separation in the Waveform Domain," *arXiv preprint arXiv:1911.13254*, 2021. [Online]. Available: <https://arxiv.org/abs/1911.13254>
- [2] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *Google AI Technical Report*, December 2023. [Online]. Available: <https://deepmind.google/technologies/gemini/>
- [3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024-8035.
- [4] S. Ramirez and S. Valente, "FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python 3.7+ Based on Standard Python Type Hints," *FastAPI Documentation*, 2023. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] Meta AI Research, "Demucs: Music Source Separation," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/facebookresearch/demucs>
- [6] International Telecommunication Union, "Recommendation ITU-R BS.1770-4: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level," *ITU-R Standards*, October 2015.
- [7] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," in *Proceedings of the 14th Python in Science Conference*, 2015, pp. 18-25. DOI: 10.25080/Majora-7b98e3ed-003
- [8] V. R. Algazi, R. O. Duda, D. M. Thompson, and C. Avendano, "The CIPIC HRTF Database," in *Proceedings of the 2001 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, New Paltz, NY, USA, 2001, pp. 99-102. DOI: 10.1109/ASPAA.2001.969552
- [9] J. Blauert, *Spatial Hearing: The Psychophysics of Human Sound Localization*, Revised Edition, MIT Press, Cambridge, MA, 1997.
- [10] Vercel Inc., "Next.js 15 Documentation: The React Framework for Production," *Next.js Official Documentation*, 2024. [Online]. Available: <https://nextjs.org/docs>
- [11] Meta Platforms Inc., "React 19: A JavaScript Library for Building User Interfaces," *React Documentation*, 2024. [Online]. Available: <https://react.dev/>
- [12] S. Choi, W. Kim, S. Park, and S. Yoon, "Temporal Modeling Matters: A Novel Temporal Emotional Modeling Approach for Speech Emotion Recognition," in *2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Singapore, 2022, pp. 6714-6718. DOI: 10.1109/ICASSP43922.2022.9746828